



Python Programming Fundamentals

CSV Files

Programming Fundamentals Team

SETU

2026-09-14



Outline

1. What is CSV?
2. Reading CSV with the csv Module
3. Writing CSV
4. DictReader — Rows as Dictionaries
5. DictWriter — Write from Dicts
6. Practical Example — Student Analysis



Outline

1. What is CSV?
2. Reading CSV with the csv Module
3. Writing CSV
4. DictReader — Rows as Dictionaries
5. DictWriter — Write from Dicts
6. Practical Example — Student Analysis



1. What is CSV?

CSV (Comma-Separated Values) is a simple plain-text format for tabular data.

```
name,age,city,score  
Alice,22,Dublin,88  
Bob,25,Cork,74  
Carol,21,Galway,95  
Dave,23,Limerick,62
```



1. What is CSV?



1. What is CSV?

CSV (Comma-Separated Values) is a simple plain-text format for tabular data.

```
name,age,city,score
Alice,22,Dublin,88
Bob,25,Cork,74
Carol,21,Galway,95
Dave,23,Limerick,62
```

Why CSV?

- Universal — opens in Excel, Google Sheets, any text editor
- Simple — no special software needed
- Lightweight — just text
- Supported by virtually every programming language and database



1. What is CSV?



1. What is CSV?

CSV (Comma-Separated Values) is a simple plain-text format for tabular data.

```
name,age,city,score
Alice,22,Dublin,88
Bob,25,Cork,74
Carol,21,Galway,95
Dave,23,Limerick,62
```

Why CSV?

- Universal — opens in Excel, Google Sheets, any text editor
- Simple — no special software needed
- Lightweight — just text
- Supported by virtually every programming language and database



1. What is CSV?

CSV files may use other delimiters: `;`, `\t` (tab), `|`. The Python `csv` module handles all of these.



Outline

1. What is CSV?
- 2. Reading CSV with the csv Module**
3. Writing CSV
4. DictReader — Rows as Dictionaries
5. DictWriter — Write from Dicts
6. Practical Example — Student Analysis



2. Reading CSV with the csv Module

```
import csv

with open("students.csv", "r", newline="") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)    # each row is a list of strings
```

Output:

```
['name', 'age', 'city', 'score']
['Alice', '22', 'Dublin', '88']
['Bob', '25', 'Cork', '74']
```



2. Reading CSV with the csv Module



2. Reading CSV with the csv Module

```
import csv

with open("students.csv", "r", newline="") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)    # each row is a list of strings
```

Output:

```
['name', 'age', 'city', 'score']
['Alice', '22', 'Dublin', '88']
['Bob', '25', 'Cork', '74']
```

Skip the header row:



2. Reading CSV with the csv Module

```
with open("students.csv", "r", newline="") as f:
    reader = csv.reader(f)
    header = next(reader)    # consume the first row
    for row in reader:
        name, age, city, score = row
        print(f"{name} from {city}: {score}")
```



Outline

1. What is CSV?
2. Reading CSV with the csv Module
- 3. Writing CSV**
4. DictReader — Rows as Dictionaries
5. DictWriter — Write from Dicts
6. Practical Example — Student Analysis



3. Writing CSV

```
import csv

students = [
    ["name", "age", "city", "score"], # header
    ["Alice", 22, "Dublin", 88],
    ["Bob", 25, "Cork", 74],
    ["Carol", 21, "Galway", 95],
]

with open("output.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(students) # write all rows at once

print("CSV written successfully")
```



3. Writing CSV



3. Writing CSV

```
import csv

students = [
    ["name", "age", "city", "score"], # header
    ["Alice", 22, "Dublin", 88],
    ["Bob", 25, "Cork", 74],
    ["Carol", 21, "Galway", 95],
]

with open("output.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(students) # write all rows at once

print("CSV written successfully")
```



3. Writing CSV

Write row by row:

```
with open("log.csv", "a", newline="") as f:  
    writer = csv.writer(f)  
    writer.writerow(["2024-01-15", "Alice", "Login", "Success"])
```



Outline

1. What is CSV?
2. Reading CSV with the csv Module
3. Writing CSV
- 4. DictReader — Rows as Dictionaries**
5. DictWriter — Write from Dicts
6. Practical Example — Student Analysis



4. DictReader — Rows as Dictionaries

`csv.DictReader` maps each row to a dict using the header row as keys.

```
import csv

with open("students.csv", "r", newline="") as f:
    reader = csv.DictReader(f)
    for student in reader:
        # student is a dict: {'name': 'Alice', 'age': '22', ...}
        name = student["name"]
        score = int(student["score"]) # convert from string!
        grade = "Pass" if score >= 60 else "Fail"
        print(f"{name}: {score} -> {grade}")
```



4. DictReader — Rows as Dictionaries



4. DictReader — Rows as Dictionaries

`csv.DictReader` maps each row to a dict using the header row as keys.

```
import csv

with open("students.csv", "r", newline="") as f:
    reader = csv.DictReader(f)
    for student in reader:
        # student is a dict: {'name': 'Alice', 'age': '22', ...}
        name = student["name"]
        score = int(student["score"]) # convert from string!
        grade = "Pass" if score >= 60 else "Fail"
        print(f"{name}: {score} -> {grade}")
```

Load all records into a list:



4. DictReader — Rows as Dictionaries

```
with open("students.csv", "r", newline="") as f:
    reader = csv.DictReader(f)
    students = list(reader)    # list of dicts

# Now you can work with the data
scores = [int(s["score"]) for s in students]
print(f"Average score: {sum(scores)/len(scores):.1f}")
```



Outline

1. What is CSV?
2. Reading CSV with the csv Module
3. Writing CSV
4. DictReader — Rows as Dictionaries
- 5. DictWriter — Write from Dicts**
6. Practical Example — Student Analysis



5. DictWriter — Write from Dicts

```
import csv

students = [
    {"name": "Alice", "age": 22, "city": "Dublin", "score": 88},
    {"name": "Bob", "age": 25, "city": "Cork", "score": 74},
    {"name": "Carol", "age": 21, "city": "Galway", "score": 95},
]

fields = ["name", "age", "city", "score"]

with open("students_out.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()          # write the header row
    writer.writerows(students)    # write all data rows
```



Outline

1. What is CSV?
2. Reading CSV with the csv Module
3. Writing CSV
4. DictReader — Rows as Dictionaries
5. DictWriter — Write from Dicts
- 6. Practical Example — Student Analysis**



6. Practical Example — Student Analysis

```
import csv

def load_students(filename):
    students = []
    with open(filename, "r", newline="") as f:
        for row in csv.DictReader(f):
            students.append({
                "name": row["name"],
                "score": int(row["score"]),
                "city": row["city"],
            })
    return students

def print_report(students):
    scores = [s["score"] for s in students]
```



6. Practical Example — Student Analysis

```
print(f"Students: {len(students)}")
print(f"Average: {sum(scores)/len(scores):.1f}")
print(f"Highest: {max(scores)}")
print(f"Lowest: {min(scores)}")
print("\nFailed students:")
for s in students:
    if s["score"] < 60:
        print(f" {s['name']} ({s['score']})")

students = load_students("students.csv")
print_report(students)
```



Thanks for Watching - Any questions?

